# Chapter 18 | Exercise solutions with Python

*Language Understanding*

This notebook solves Exercises 5–8. It checks masked pooling, word-to-piece alignment, IOB2 repair, and exact matching of typed entity spans. Exercises 1–4 and 9–11 are in the written solutions.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements.txt`. Run the cells in order. The exercises run on CPU and include their saved outputs.

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch18_understanding"
    )
    if model_path.is_dir():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

## 5. Change the padding rows

The first three rows from Exercise 2 represent tokens in a sentence. The fourth row is padding. To make a sentence prediction, the head first pools the token features, meaning it combines them into one vector. In masked mean pooling, `True` selects a row for the average and `False` leaves it out of both the sum and the count. With the Boolean mask `(True, True, True, False)`, the hand calculation is

$$\frac{(2, 0) + (1, 3) + (0, 1)}{3} = \left(1, \frac{4}{3}\right).$$

Each logit is a raw class score. The linear head takes the dot product of the pooled vector with one row of weights per class and adds that class's bias. Here `np.eye(2)` gives the identity matrix and `np.zeros(2)` gives zero biases, so the first logit is the first pooled coordinate and the second logit is the second. Replacing the fourth row $(6, 6)$ with $(20, -10)$ leaves the same three rows selected. We therefore expect the same pooled vector and logits before and after the replacement.

```
from examples.ch18_understanding.model import sentence_logits

features = np.array([[2, 0], [1, 3], [0, 1], [6, 6]])
valid = np.array([True, True, True, False])
weight = np.eye(2)
bias = np.zeros(2)

changed_features = features.copy()
changed_features[-1] = [20, -10]
expected_masked = [1, 4 / 3]

for name, rows in [("Original", features),
                   ("Changed padding", changed_features)]:
    pooled, logits = sentence_logits(rows, valid, weight, bias)
    shown_pool = pooled.round(6)
    shown_logits = logits.round(6)
    print(f"{name:16} pooled {shown_pool}, logits {shown_logits}")
    np.testing.assert_allclose(pooled, expected_masked)
    np.testing.assert_allclose(logits, expected_masked)
```

```
OUTPUT
Original         pooled [1.       1.333333], logits [1.       1.333333]
Changed padding  pooled [1.       1.333333], logits [1.       1.333333]
```

Marking the last row `True` makes the mask select all four rows, so the helper now computes the ordinary unmasked mean. The sum of the first three rows is $(3, 4)$. With the original fourth row, the mean would be $((3+6)/4, (4+6)/4) = (9/4, 5/2)$. After the change, it is

$$\frac{(3,4)+(20,\,-10)}{4} = \left(\frac{23}{4}, -\frac{3}{2}\right).$$

Both coordinates can change because the fourth row now contributes to the sum, and the divisor is four. The identity head copies each new mean directly into the logits.

```
all_valid = np.ones_like(valid)
unmasked_cases = [
    ("Original", features, [9 / 4, 5 / 2]),
    ("Changed padding", changed_features, [23 / 4, -3 / 2]),
]
for name, rows, expected_mean in unmasked_cases:
    pooled, logits = sentence_logits(rows, all_valid, weight, bias)
    shown_pool = pooled.round(6)
    shown_logits = logits.round(6)
    print(f"{name:16} pooled {shown_pool}, logits {shown_logits}")
    np.testing.assert_allclose(pooled, expected_mean)
    np.testing.assert_allclose(logits, expected_mean)
```

```
OUTPUT
Original         pooled [2.25 2.5 ], logits [2.25 2.5 ]
Changed padding  pooled [ 5.75 -1.5 ], logits [ 5.75 -1.5 ]
```

## 6. Align a split word

There are three annotated words and seven token positions. A word ID tells us which word a token piece belongs to. The ID `-1` marks a position with no word, and consecutive pieces of the same word share its ID. In `(-1, 0, 0, 1, 2, 2, -1)`, words 0 and 2 each have two pieces. Word 1 has one piece.

With first-piece supervision, copy the label of each word to its first piece. Give the other positions the target `-100`, the default ignore value, so their own predictions contribute no direct loss. At evaluation time, read the predicted class IDs at the same first-piece positions to get one prediction per word. The table uses zero-based token positions.

| Token position | Word ID | Training target | Prediction | Keep? |
|---|---|---|---|---|
| 0 | -1 | -100 | 9 | No, no word |
| 1 | 0 | 4 | 4 | Yes, first piece |
| 2 | 0 | -100 | 8 | No, continuation |
| 3 | 1 | 0 | 0 | Yes, first piece |
| 4 | 2 | 2 | 2 | Yes, first piece |
| 5 | 2 | -100 | 7 | No, continuation |
| 6 | -1 | -100 | 9 | No, no word |

```
from examples.ch18_understanding.model import (
    IGNORE,
    first_piece_labels,
    first_piece_predictions,
)

word_ids = np.array([-1, 0, 0, 1, 2, 2, -1])
word_labels = np.array([4, 0, 2])
token_predictions = np.array([9, 4, 8, 0, 2, 7, 9])

token_targets = first_piece_labels(word_ids, word_labels)
word_predictions = first_piece_predictions(word_ids, token_predictions)
kept_positions = np.flatnonzero(token_targets != IGNORE)
ignored_positions = np.flatnonzero(token_targets == IGNORE)
ignored_predictions = token_predictions[ignored_positions]

print("Token targets", token_targets.tolist())
print("Kept token positions", kept_positions.tolist())
print("Word predictions", word_predictions.tolist())
print("Ignored token positions", ignored_positions.tolist())
print("Ignored token predictions", ignored_predictions.tolist())

np.testing.assert_array_equal(
    token_targets, [-100, 4, -100, 0, 2, -100, -100]
)
np.testing.assert_array_equal(kept_positions, [1, 3, 4])
np.testing.assert_array_equal(word_predictions, [4, 0, 2])
np.testing.assert_array_equal(ignored_positions, [0, 2, 5, 6])
np.testing.assert_array_equal(ignored_predictions, [9, 8, 7, 9])
```

```
OUTPUT
Token targets [-100, 4, -100, 0, 2, -100, -100]
Kept token positions [1, 3, 4]
Word predictions [4, 0, 2]
Ignored token positions [0, 2, 5, 6]
Ignored token predictions [9, 8, 7, 9]
```

The three retained class IDs match their respective word labels. Reading first pieces gives a direct, word-by-word comparison with the original annotations.

## 7. Choose a repair policy

Read `B-LOC, I-LOC, I-PER, I-PER, O` one word at a time. In the chapter's IOB2 convention, `B-LOC` starts a location, `I-LOC` continues an open location, and `O` marks a word outside any entity. The first two tags open and extend a location. The next tag, `I-PER`, says to continue a person, but the open entity is a location. That is the first invalid continuation.

The default decoder stops at the invalid tag. Catch the exception so that the notebook records the failure and can continue to the repair call.

```
from examples.ch18_understanding.model import bio_spans

repair_tags = np.array(["B-LOC", "I-LOC", "I-PER", "I-PER", "O"])

try:
    bio_spans(repair_tags)
except ValueError as error:
    assert str(error) == "unexpected inside tag at word 2: I-PER"
    print(f"{type(error).__name__}: {error}")
else:
    raise AssertionError("Strict decoding should reject I-PER.")
```

```
OUTPUT
ValueError: unexpected inside tag at word 2: I-PER
```

Word index 2 is the **third** word because indexing starts at zero. With `repair=True`, the first `I-PER` acts as a new start at index 2. The decoder closes the location at that same index and saves it before opening the person span. The next `I-PER` continues the person, and `O` at index 4 closes it.

A typed span records its start, its exclusive end, and its entity type. The half-open interval $[0, 2)$ includes word indices 0 and 1, just as the Python slice `[0:2]` does. The person covers $[2, 4)$, so the two entities are adjacent and share no words.

```
repaired_spans = bio_spans(repair_tags, repair=True)

assert repaired_spans == {(0, 2, "LOC"), (2, 4, "PER")}
for start, end, entity in sorted(repaired_spans):
    included = repair_tags[start:end].tolist()
    print(f"{entity} [{start}, {end}) covers {included}")
```

```
OUTPUT
LOC [0, 2) covers ['B-LOC', 'I-LOC']
PER [2, 4) covers ['I-PER', 'I-PER']
```

Set the prediction repair policy before scoring and keep gold annotations in their stated convention.

## 8. Test a boundary error (optional)

Use `O, B-LOC, I-LOC, O, B-PER` as the gold word tags. There are two gold entities. The location covers indices 1 and 2, and the person covers index 4. Make each prediction from a fresh copy of gold so that the three changes are independent.

| Prediction | Word index | Gold tag | Predicted tag |
|---|---|---|---|
| Shorter location | 2 | `I-LOC` | `O` |
| Wrong type | 4 | `B-PER` | `B-ORG` |
| Extra entity | 0 | `O` | `B-ORG` |

All three sequences follow IOB2. A one-word entity begins with `B`, so the organization at index 0 in the third prediction can appear immediately before the location.

```
gold_tags = np.array(["O", "B-LOC", "I-LOC", "O", "B-PER"])
changes = (
    ("Shorter location", 2, "O"),
    ("Wrong type", 4, "B-ORG"),
    ("Extra entity", 0, "B-ORG"),
)

gold_entities = bio_spans(gold_tags)
assert gold_entities == {(1, 3, "LOC"), (4, 5, "PER")}
print("Gold spans", sorted(gold_entities))

predictions = {}
for name, index, new_tag in changes:
    tags = gold_tags.copy()
    tags[index] = new_tag
    np.testing.assert_array_equal(
        np.flatnonzero(tags != gold_tags), [index]
    )
    predictions[name] = tags
    print(f"{name:<17} {', '.join(tags)}")
```

```
OUTPUT
Gold spans [(1, 3, 'LOC'), (4, 5, 'PER')]
Shorter location  O, B-LOC, O, O, B-PER
Wrong type        O, B-LOC, I-LOC, O, B-ORG
Extra entity      B-ORG, B-LOC, I-LOC, O, B-PER
```

Exact matching requires the start, end, and type to agree. Decode the predictions so that we can inspect the full triples. The intersection of the predicted and gold sets contains the exact matches. Let $M$ be the number of matches, $N_p$ the number of predictions, and $N_g$ the number of gold entities. Precision divides by predicted spans and recall divides by gold spans.

$$P = \frac{M}{N_p}, \qquad R = \frac{M}{N_g}, \qquad F_1 = \frac{2M}{N_p + N_g}.$$

Use `span_metrics` for each prediction. Check the results against the expected exact matches and the counts implied by the three tag changes.

```
from examples.ch18_understanding.model import span_metrics

expected = {
    "Shorter location": ({(4, 5, "PER")}, 2, 1 / 2, 1 / 2, 1 / 2),
    "Wrong type": ({(1, 3, "LOC")}, 2, 1 / 2, 1 / 2, 1 / 2),
    "Extra entity": (gold_entities, 3, 2 / 3, 1, 4 / 5),
}
```

```
for name, tags in predictions.items():
    predicted_entities = bio_spans(tags)
    matched = predicted_entities & gold_entities
    scores = span_metrics(gold_tags, tags)
    matches, count, precision, recall, f1 = expected[name]

    assert matched == matches
    assert scores["correct_spans"] == len(matches)
    assert scores["predicted_spans"] == count
    assert scores["gold_spans"] == 2
    np.testing.assert_allclose(
        [scores["precision"], scores["recall"], scores["f1"]],
        [precision, recall, f1],
    )
    assert scores["word_accuracy"] == 4 / 5

    print(name)
    print(f"  Predicted  {sorted(predicted_entities)}")
    print(f"  Matched    {sorted(matched)}")
    print(f"  Counts     matched {scores['correct_spans']}, "
          f"predicted {count}, gold {scores['gold_spans']}")
    print(f"  Precision {scores['precision']:.3f}   "
          f"Recall {scores['recall']:.3f}   "
          f"F1 {scores['f1']:.3f}")
```

OUTPUT

```
Shorter location
  Predicted  [(1, 2, 'LOC'), (4, 5, 'PER')]
  Matched    [(4, 5, 'PER')]
  Counts     matched 1, predicted 2, gold 2
  Precision 0.500   Recall 0.500   F1 0.500
Wrong type
  Predicted  [(1, 3, 'LOC'), (4, 5, 'ORG')]
  Matched    [(1, 3, 'LOC')]
  Counts     matched 1, predicted 2, gold 2
  Precision 0.500   Recall 0.500   F1 0.500
Extra entity
  Predicted  [(0, 1, 'ORG'), (1, 3, 'LOC'), (4, 5, 'PER')]
  Matched    [(1, 3, 'LOC'), (4, 5, 'PER')]
  Counts     matched 2, predicted 3, gold 2
  Precision 0.667   Recall 1.000   F1 0.800
```

- **Shorter location.** The predicted location $(1, 2)$ stops before word index 2, while the gold location $(1, 3)$ includes it. The person is the only match. There is one match out of two predictions and two gold spans, so both precision and recall are $1/2$.
- **Wrong type.** The entity at $(4, 5)$ has the right boundaries but is predicted as `ORG`. Only the location matches. Precision and recall are again $1/2$.
- **Extra entity.** The organization at $(0, 1)$ adds a false positive. Both gold entities still match, so recall is $2/2 = 1$ and precision is $2/3$.

All three predictions have word accuracy $4/5$ because each changes exactly one of five tags. That single score cannot tell whether a prediction missed an entity or added one.